

Workshop on Advanced Sampling Methodologies

Simulation and optimization of sampling strategies

Dr. Gianluca Boo, WorldPop, University of Southampton

2025-09-24

Workshop's agenda

- **Day 1:** Introduction to QGIS
- **Day 2:** Automatic update of census EA and national sampling frame
- **Day 3:** Introduction to R programming
- **Day 4:** Introduction to Sampling Design
- **Day 5:** Advanced R programming
- **Day 6:** Earth observation (EO) for sampling design
- **Day 7:** AI and machine learning for sample selection
- **Day 8: Simulation and optimization of sampling strategies**
- **Day 9:** R Shiny for sampling design evaluation
- **Day 10:** Wrap-up and workshop evaluation

Course materials

1. Today's course materials are on **Github**:
https://github.com/gianlucaboo/sampling_design_workshop
2. Click on **Day 8: Simulation and optimization of sampling strategies**
3. **Copy the url** and **paste it** on <https://download-directory.github.io>
4. Save the zipped directory on the workshop and unzip it

Today's agenda

- Law of Large Numbers and Central Limit theorem
- Cochran's formula versus Monte Carlo simulation
- Sampling distributions, mean, standard error
- Design comparisons (SRS, stratified, cluster)
- Practical exercises

Sampling design optimization

Field surveys are expensive and require designs that are **accurate** and **efficient**. – A crucial parameter is **sample size**, which is generally computed using **Cochran’s formula**.

– Monte Carlo simulations let us evaluate expected performance offers a flexible alternative.

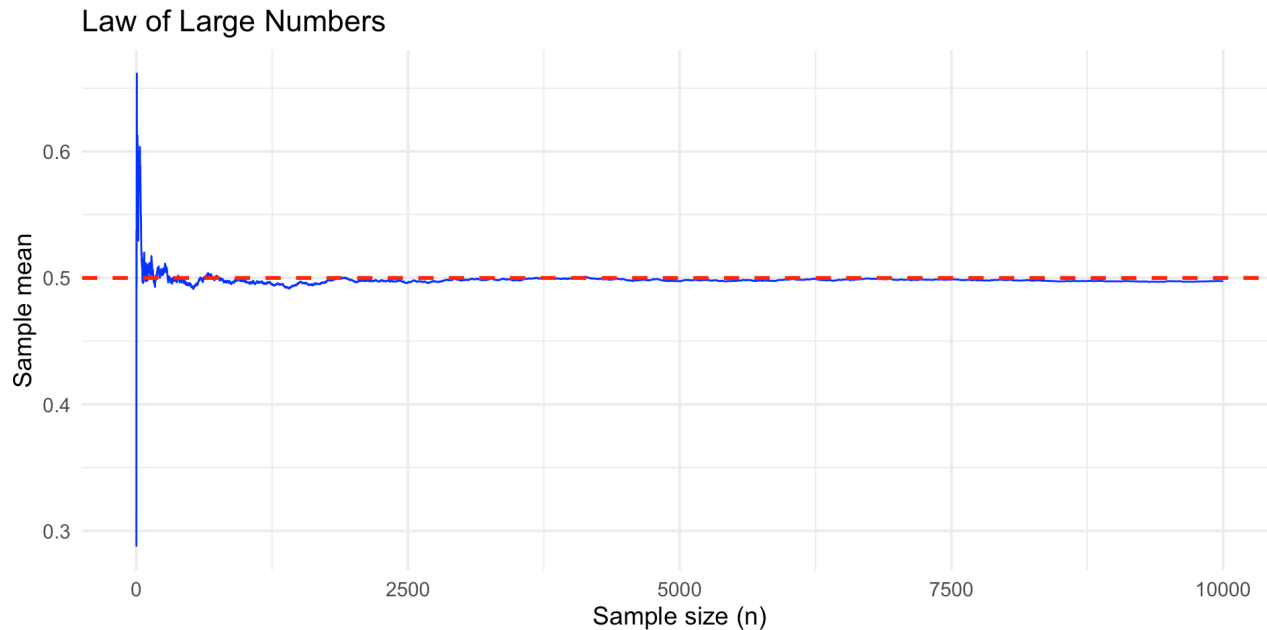
The main **characteristics of these two approaches** are summarised below:

	Cochran’s Formula	Monte Carlo Simulation
Purpose	Quick sample size determination under SRS.	Evaluate performance under any sampling design.
Method	Closed statistical formula.	Computational, repeat simulations.
Assumptions	Simple random sampling.	Works with stratified, cluster, PPS, multi-stage designs.
Outputs	Required sample size <i>n</i> .	Bias, variance, RMSE, CI, precision.
Complexity	Simple to apply (pen-and-paper).	Requires coding and computational power.
Accuracy	May fail for complex designs or small populations.	High fidelity as it mimics actual sampling process.

Law of Large Numbers

The **sample mean** ($\bar{X}_n$) of independent, identically distributed (i.i.d.) random variables (X) converges to the **population mean** (μ) as the sample size (n) grows.

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \rightarrow \mu \text{ as } n \rightarrow \infty$$



Law of Large Numbers in R

The **Law of Large Numbers** can be demonstrated in R.

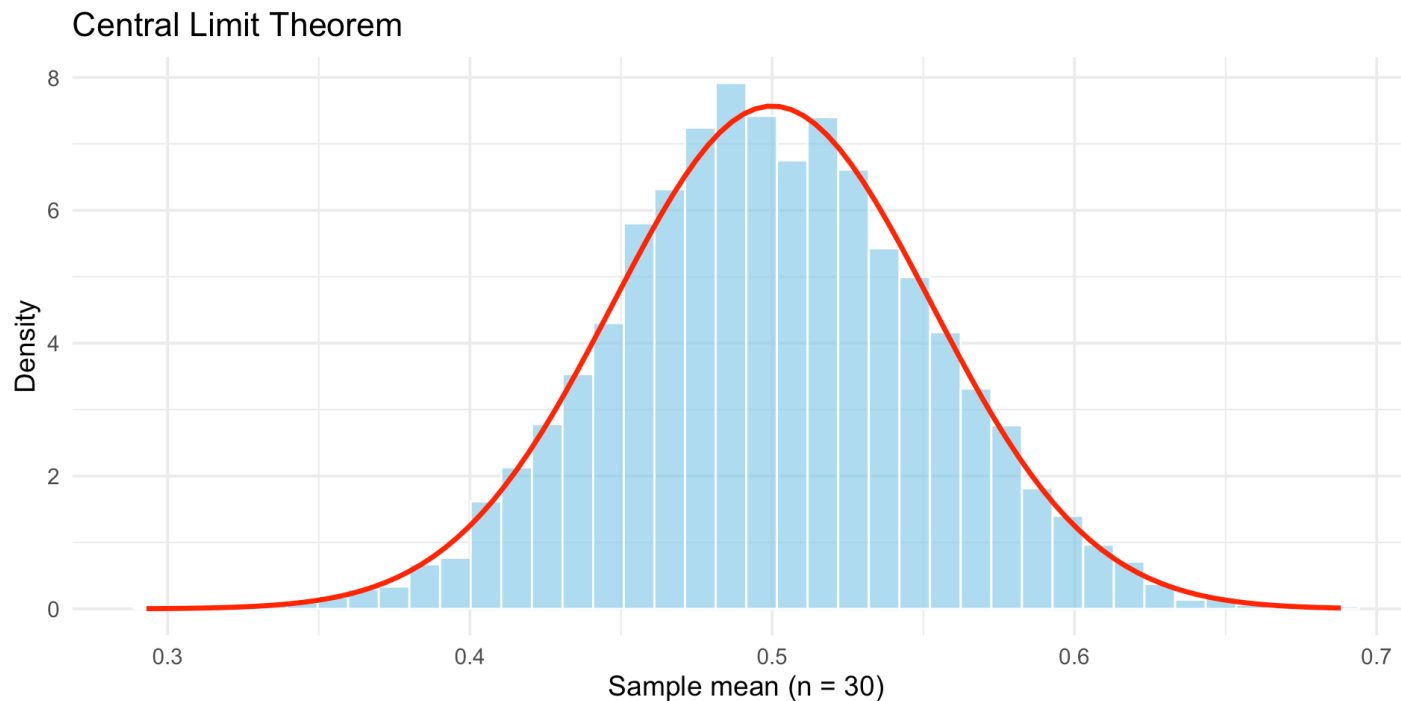
```
1 library(tidyverse)
2 set.seed(123)                # reproducibility
3 n <- 10000                    # total number of draws
4 x <- runif(n, min = 0, max = 1) # random draws from Uniform(0,1)
5
6 true_mean <- 0.5              # theoretical mean of U(0,1)
7
8 # compute cumulative means
9 cum_means <- cumsum(x) / (1:n)
10
11 df <- data.frame(
12   n = 1:n,
13   cum_mean = cum_means
14 )
15
16 # Plot convergence
17 ggplot(df, aes(x = n, y = cum_mean)) +
18   geom_line(color = "blue") +
```

 Try to run the code in your Script Editor.

Central Limit theorem

The distribution of the **sample mean** approaches a **normal distribution** as the sample size (n) increases, regardless of the population distribution.

$$\sqrt{n} (\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$$



Central Limit theorem in R

Again, the **Central Limit theorem** can be demonstrated in R.

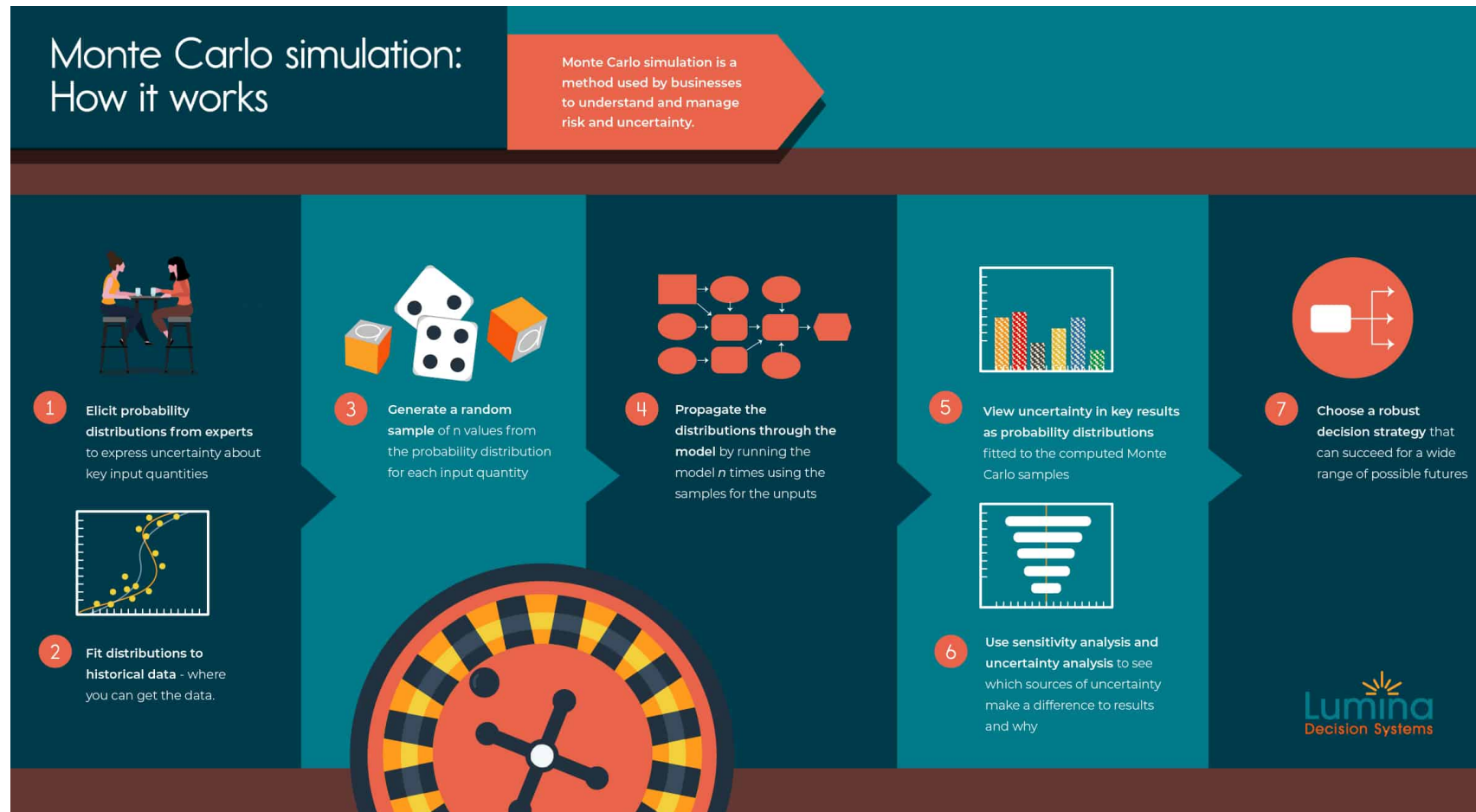
```
1 library(ggplot2)
2
3 set.seed(123)
4
5 # Parameters
6 n <- 30          # sample size per experiment
7 n_sim <- 5000    # number of simulations
8
9 # Repeat experiments: sample from Uniform(0,1), compute sample mean
10 means <- replicate(n_sim, mean(runif(n, 0, 1)))
11
12 df_means <- data.frame(sample_mean = means)
13
14 true_mean <- 0.5
15 true_sd <- 1 / sqrt(12)      # SD of U(0,1)
16 se <- true_sd / sqrt(n)     # standard error of sample mean
17
18 # Plot distribution of sample means
```



Try to run the code in your Script Editor.

Monte Carlo simulation

A Monte Carlo simulation involves **repeated random sampling** from a known population model to approximate the **sampling distribution of estimators**.



Monte Carlo simulation workflow (source: Lumina)

Monte Carlo simulation

Monte Carlo simulations can be **evaluated using different metrics** depending on the purpose of the analysis. The most common metrics are:

- **Bias** = $\text{average}(\text{estimate}) - \text{true value}$
- **Variance** = $\text{Var}(\text{estimate})$
- **RMSE** = $\sqrt{\text{Bias}^2 + \text{Variance}}$
- **Empirical SE** = $\text{sd}(\text{estimate})$
- **Coverage** of nominal 95% CI = proportion of reps where CI contains true value

 These parametric metrics can be complemented by non-parametric statistics, such as the **Empirical Cumulative Distribution function** (ECDF) and the **Kologomorov-Smirnov** statistics (see Boo et al. 2021).

Monte Carlo simulation in R

A Monte Carlo simulation demonstration in R for **simple random sampling** with sample size $n=30$.

```
1 library(ggplot2)
2
3 set.seed(123)
4
5 # Population model:  $X \sim \text{Normal}(\mu=5, \sigma=2)$ 
6 mu <- 5
7 sigma <- 2
8
9 n <- 30          # sample size
10 R <- 2000        # number of Monte Carlo replicates
11
12 # Monte Carlo loop: compute sample mean each time
13 means <- replicate(R, mean(rnorm(n, mean = mu, sd = sigma)))
14
15 df <- data.frame(estimates = means)
16
17 # Plot the distribution of sample mean estimates
18 ggplot(df, aes(x = estimates)) +
```

 Try to run the code in your Script Editor.

Monte Carlo simulation in R

A Monte Carlo simulation for different sample size assessing the **empirical Standard Error of the estimator** for each replicate, assuming that $SE(\bar{\mu}) \approx \sqrt{\text{Var}(\bar{\mu})} = \frac{\sigma}{\sqrt{n}}$.

```
1 library(ggplot2); library(dplyr)
2
3 set.seed(123)
4
5 mu <- 5
6 sigma <- 2
7 R <- 2000 # number of Monte Carlo replicates
8 sample_sizes <- c(1:100) # try different n
9
10 # Run Monte Carlo for each sample size
11 results <- lapply(sample_sizes, function(n) {
12   means <- replicate(R, mean(rnorm(n, mean = mu, sd = sigma)))
13   data.frame(
14     n = n,
15     mean_est = mean(means),
16     se_empirical = sd(means) # empirical standard error
17   )
18 }) |> bind rows()
```

 Try to run the code in your Script Editor.

Monte Carlo simulation in R

A Monte Carlo simulation for different sample size assessing the **95% CI coverage** using $CI_{0.95} = \bar{X} \pm z_{0.975} \frac{s}{\sqrt{n}}$ for each replicate, with $CI_{0.95} \approx 1.96$ for a 95% CI.

```
1 library(dplyr); library(ggplot2)
2
3 set.seed(123)
4
5 mu <- 5
6 sigma <- 2
7 R <- 2000
8 sample_sizes <- 1:100
9
10 results <- lapply(sample_sizes, function(n) {
11   coverages <- replicate(R, {
12     samp <- rnorm(n, mean = mu, sd = sigma)
13     xbar <- mean(samp)
14     s <- sd(samp)
15     se <- s / sqrt(n)
16
17     # 95% CI using normal approximation
18     lower <- xbar - 1.96 * se
```

 Try to run the code in your Script Editor.

Monte Carlo simulation in R

For **stratified sampling**, the population is divided in **three strata**. Two allocation rules are tested: **proportional** and **Neyman** (stratum size $\times$ stratum standard deviation).

```
1 library(dplyr); library(purrr); library(tibble)
2 set.seed(1) # Fix random seed for reproducibility
3 R <- 1000 # Number of Monte Carlo replicates
4 N_h <- c(4000, 3000, 3000) # Stratum population sizes (3 strata)
5 H <- length(N_h) # Number of strata
6 N <- sum(N_h) # Total population size
7
8 # Create artificial stratified population. Each unit belongs to a stratum h
9 pop <- tibble(h = rep(1:H, times = N_h)) %>%
10   mutate(y = case_when(
11     h==1 ~ rnorm(n(), 50, 10), # Stratum 1: mean = 50, sd = 10
12     h==2 ~ rnorm(n(), 60, 20), # Stratum 2: mean = 60, sd = 20
13     h==3 ~ rnorm(n(), 55, 12) # Stratum 3: mean = 55, sd = 12
14   ))
15
16 true_mean <- mean(pop$y) # True population mean across all strata
17
18 #Define simulation function
```

Monte Carlo simulation in R

It is useful to create **functions replicating different sampling designs**, with **arguments representing the sampling parameters**. These can then be tested through extensive simulations.

```
1 library(purrr)
2
3 res_prop <-
4 c(1000:1005) %>% # sample size from 200 to 250
5 map(function(i) {
6   sim <- simulate_alloc("prop", n=i)
7
8   data.frame(
9     type = "alloc",
10    n=i,
11    mean=mean(sim$ybar),
12    se=sd(sim$ybar)
13  )}
14 ) %>%
15 list_rbind()
16
17 # Standard error vs sample size
18 ggplot(res_prop, aes(x = n, y = se)) +
```

Parallel computation in R

Monte Carlo simulations require **many independent replicates**, each typically running independently. Parallel computation **reduce computation time** significantly using multi-core CPUs or HPC.

```
1 library(furrr)
2
3 set.seed(123) # master seed
4
5 plan(multisession, workers = 4) # use 4 cores increase speed ~x4
6
7 res_prop <-
8 c(1000:1005) %>% # sample size from 200 to 250
9 future_map(function(i) {
10   sim <- simulate_alloc("prop", n=i)
11
12   data.frame(
13     type = "alloc",
14     n=i,
15     mean=mean(sim$ybar),
16     se=sd(sim$ybar)
17   )},
18 .options = furrr_options(seed = TRUE)
```

Parallel computation in R

Parallel computation should be **used carefully** because a wrong set up can severely impact performance. Key elements to consider are:

Choice of packages

- `parallel` (base R, `mclapply`, `parLapply`)
- `foreach` + `%dopar%` (flexible backends, e.g. `doParallel`)
- `future` + `furrr` (tidyverse-friendly, reproducible)

Number of cores

- Detect with `parallel::detectCores()` or `future::availableCores()`
- Often safer to use $N-1$ cores to leave resources for system tasks

Simulation size

- Larger `R` (replicates) benefits more from parallelisation
- Small tasks may not offset communication overhead

Parallel computation in R

Computation time

- Use timing tools to compare sequential vs parallel using the library `tictoc`

Reproducibility

- Always control random number generation
- `set.seed(123)` for sequential
- `.options = furrr_options(seed = TRUE)` for `furrr`

Scalability

- Performance depends on hardware and task complexity.
- Benchmark before running very large simulations

Take home

- **Cochran's formula is quick but limited** as efficient for simple random sampling but less accurate for complex designs.
- **Monte Carlo simulations are flexible** and allow evaluation of any sampling design (stratified, cluster, PPS, multi-stage).
- **Law of Large Numbers and CLT** ensure sample means converge to population means and approximate normality for inference.
- **Optimization is crucial** to balance precision and cost through allocation strategies (e.g., proportional vs. Neyman).
- **Parallel computing accelerates research** but reproducibility, number of cores, and simulation size must be carefully managed.

Resources

- Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley.
- Lohr, S. (2019). *Sampling: Design and Analysis* (2nd ed.). CRC Press.
- Boo, G., et al. (2021). A grid-based sample design framework for household surveys. *Gates Open Research*. <https://doi.org/10.12688/gatesopenres.13107.1>
- Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341. <https://doi.org/10.1080/01621459.1949.10483310>
- McCallum, A. (2012). *Parallel R: Data Analysis in the Distributed World*. O'Reilly Media.
- Vaughan, D., & Dancho, M. (2024). *furrr: Apply Mapping Functions in Parallel using Futures*. <https://furrr.futureverse.org>

Exercise

Please download the R script with exercises from **GitHub** and try to complete it.